RoBERTa Two-Stage Pipeline

IT Service Desk Ticket Classification | Comprehensive Test Case Suite

Covers: Data Ingestion · Stage 1 MLM Pretraining · Data Splitting · Stage 2 Fine-Tuning · Evaluation · Inference · Pipeline Integration · Security · Performance · Robustness

Test Suite Summary

Summary Metric	Count
Total Test Cases	114
Modules / Sections Covered	10
Critical Priority	33
High Priority	52
Medium Priority	26
Low Priority	3
Positive Tests	93
Negative Tests	10
Security / Data-Leakage Tests	3
Performance Tests	8

Full Test Case Register

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
Data Ingestion (TC001 – TC015)							
TC ID	**Module**	**Feature**	**Description**	**Test Steps**	**Expected Result**	**Priority**	**Type**
TC001	Data Ingestion	CSV Loading	Load well-formed CSV with all required columns (ticket_description, ticket_category)	1. Prepare CSV with ticket_description and ticket_category columns 2. Call load_labeled_data() 3. Inspect returned DataFrame	DataFrame loaded with correct dtypes; row count matches file; no errors raised	Critical	Positive
TC002	Data Ingestion	Multiple Source Files	prepare_unlabeled_csv() ingests and concatenates all four source CSVs	1. Pass all four SOURCE_FILES paths to prepare_unlabeled_csv() 2. Run function 3. Inspect output CSV row count	Output CSV contains union of all source files; before/after counts printed; no file-load error	Critical	Positive
TC003	Data Ingestion	Deduplication	prepare_unlabeled_csv() removes duplicate ticket_description rows across all source files	1. Include two source CSVs with 20% overlapping descriptions 2. Run prepare_unlabeled_csv() 3. Compare before vs after count in console output	Duplicates removed count > 0; output CSV has no duplicate stripped descriptions	Critical	Positive
TC004	Data Ingestion	Missing Column	load_labeled_data() raises error when ticket_category column is absent	1. Prepare CSV without ticket_category column 2. Call load_labeled_data() 3. Observe exception	KeyError or ValueError raised with message indicating missing column before processing begins	High	Negative
TC005	Data Ingestion	Missing Column	load_labeled_data() raises error when ticket_description column is absent	1. Prepare CSV without ticket_description 2. Call load_labeled_data() 3. Observe exception	Descriptive error raised; pipeline halts before any preprocessing	High	Negative
TC006	Data Ingestion	NaN Handling	Rows with NaN ticket_description dropped during labeled ingestion	1. Prepare CSV with 5 rows having NaN description 2. Call load_labeled_data() 3. Inspect cleaned DataFrame	NaN rows removed; 'loaded / after cleaning / removed' counts printed correctly	High	Positive
TC007	Data Ingestion	NaN Handling	Rows with NaN ticket_category dropped during labeled ingestion	1. Prepare CSV with 5 rows having NaN category 2. Call load_labeled_data() 3. Inspect output	Rows with missing labels removed; removal count reported in console output	High	Positive
TC008	Data Ingestion	Text Normalisation	load_labeled_data() strips and lowercases both text and label columns	1. Prepare CSV with mixed-case categories ('Hardware', 'HARDWARE') and leading/trailing spaces 2. Call load_labeled_data() 3. Inspect ticket_category values	All labels lowercased and stripped; 'Hardware' and 'HARDWARE' unified to single class 'hardware'	High	Positive
TC009	Data Ingestion	Deduplication	Duplicate (description, category) pairs removed by load_labeled_data()	1. Prepare CSV with 10 exact duplicate rows (same description + same category) 2. Call load_labeled_data() 3. Compare before vs after counts	Duplicate rows dropped; final count reduced by 10; console shows 'removed' count	High	Positive
TC010	Data Ingestion	Empty Dataset	load_labeled_data() handles CSV with headers only (zero data rows)	1. Prepare CSV with only header row 2. Call load_labeled_data() 3. Observe return value and console output	Empty DataFrame returned; informative message or count of 0 printed; no crash	Medium	Negative
TC011	Data Ingestion	UTF-8 Encoding	Unlabeled CSV with accented/UTF-8 characters loaded without corruption	1. Create CSV with descriptions containing characters such as résumé, naïve, über 2. Call load_unlabeled_texts() 3. Inspect returned list	All characters read correctly; no '?' or replacement characters; list length matches row count	Medium	Positive

TC012	Data Ingestion	Large Dataset	prepare_unlabeled_csv() handles 100,000-row input within acceptable time	1. Generate synthetic CSV with 100k ticket_description rows 2. Run prepare_unlabeled_csv() 3. Measure wall-clock time	CSV created in under 60 seconds; no OOM error; output row count correct	Medium	Performance
TC013	Data Ingestion	Empty Description	Empty string descriptions removed after strip/dedup logic in prepare_unlabeled_csv()	1. Prepare source CSV with 10 rows containing only whitespace in ticket_description 2. Run prepare_unlabeled_csv() 3. Inspect output CSV	Whitespace-only rows not present in output; 'Records saved' count excludes them	Medium	Positive
TC014	Data Ingestion	File-Not-Found	prepare_unlabeled_csv() gracefully handles a missing source file path	1. Include a non-existent file path in SOURCE_FILES list 2. Run prepare_unlabeled_csv() 3. Observe console output	Warning printed for missing file; processing continues with remaining valid files; no crash	Medium	Negative
TC015	Data Ingestion	Output Path	prepare_unlabeled_csv() saves output CSV to the specified output_path argument	1. Call prepare_unlabeled_csv() with output_path='custom_out.csv' 2. Check filesystem after function returns	File 'custom_out.csv' exists on disk; row count matches function's printed 'Records saved'	Low	Positive

Stage 1 – Domain-Adaptive MLM Pretraining (TC016 – TC028)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC016	Stage 1 MLM	Tokenizer Load	RobertaTokenizerFast loaded from roberta-base without errors	1. Call RobertaTokenizerFast.from_pretrained('roberta-base') 2. Inspect tokenizer object	Tokenizer loaded; vocab_size == 50265; no import or download errors	Critical	Positive
TC017	Stage 1 MLM	Dataset Shape	UnlabeledTicketDataset returns correct input_id tensor shape for max_length=128	1. Create UnlabeledTicketDataset with 10 texts, max_length=128 2. Access item[0] 3. Check tensor dimensions	input_ids shape [128]; attention_mask shape [128]; dtype torch.long	Critical	Positive
TC018	Stage 1 MLM	MLM Collator	DataCollatorForLanguageModeling masks ~15% of tokens per batch	1. Create batch of 8 sequences 2. Apply collator with mlm_probability=0.15 3. Count masked positions	Approximately 15% of non-padding tokens replaced with [MASK] (±5% tolerance per batch)	Critical	Positive
TC019	Stage 1 MLM	Train/Val Split	train_stage1_mlm() correctly splits unlabeled texts into train and 5% validation sets	1. Provide 1000-row unlabeled CSV 2. Call train_stage1_mlm() with val_split=0.05 3. Inspect printed train/val counts	val count = 50; train count = 950; printed message confirms counts	High	Positive
TC020	Stage 1 MLM	Loss Decreasing	Training loss decreases across 3 epochs of Stage 1	1. Run train_stage1_mlm() for 3 epochs on representative data 2. Capture printed epoch losses	Epoch 2 train loss < Epoch 1 train loss; trend indicator '↓' shown for at least one epoch	High	Positive
TC021	Stage 1 MLM	Perplexity Reporting	Epoch log line includes train PPL and val PPL values	1. Run one epoch of train_stage1_mlm() 2. Inspect console output	Log line contains 'PPL' for both train and val; values are positive floats > 1.0	High	Positive
TC022	Stage 1 MLM	Model Persistence	Stage 1 model and tokenizer saved to output_dir after training	1. Run train_stage1_mlm() with output_dir='./test_stage1' 2. List directory contents after training	config.json, pytorch_model.bin (or model.safetensors), tokenizer files present in output_dir	Critical	Positive
TC023	Stage 1 MLM	Gradient Clipping	Gradient norm clipped to 1.0 each training step to prevent exploding gradients	1. Hook into model parameters to record pre-clip norms during one epoch 2. Verify clip applied	torch.nn.utils.clip_grad_norm_ called with max_norm=1.0 each step; no NaN loss observed	High	Positive

TC024	Stage 1 MLM	Cosine Scheduler	Learning-rate cosine schedule with 10% warmup applied during Stage 1 training	1. Instrument optimizer to log lr at each step during a 3-epoch run 2. Plot or inspect lr values	LR increases linearly for first 10% of steps then follows cosine decay; never exceeds base lr	High	Positive
TC025	Stage 1 MLM	Device Placement	Model and batch tensors moved to CUDA when GPU is available	1. Run train_stage1_mlm() on machine with CUDA GPU 2. Check model.device and batch tensor devices	model.device is 'cuda'; input_ids.device is 'cuda'; no CPU-GPU transfer errors	High	Positive
TC026	Stage 1 MLM	Stall Detection	Console output shows 'X stalled' indicator when loss improvement < 0.005	1. Monkeypatch model to return constant loss 2. Run two epochs 3. Check printed trend message	'X stalled' message printed for epoch 2; drop value displayed and is <= 0.005	Medium	Positive
TC027	Stage 1 MLM	Batch Size	DataLoader produces batches of exactly batch_size samples (except last batch)	1. Run train_stage1_mlm() with batch_size=16 and 100 unlabeled texts 2. Log batch sizes in first epoch	All but possibly last batch have exactly 16 samples; no shape mismatch errors	Medium	Positive
TC028	Stage 1 MLM	Return Value	train_stage1_mlm() returns the output_dir string path	1. Capture return value of train_stage1_mlm() 2. Assert type and value	Returned value is a non-empty string equal to the output_dir argument passed in	Medium	Positive

Labeled Data Split & Class Weights (TC029 – TC038)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC029	Data Split	80/10/10 Split	train_stage2_classifier() produces 80/10/10 train/val/test split	1. Provide balanced labeled CSV with 1000 rows 2. Run train_stage2_classifier() 3. Read printed split summary	train ≈ 800 rows (80%), val ≈ 100 (10%), test ≈ 100 (10%); totals sum to 1000	Critical	Positive
TC030	Data Split	Stratified Split	Class proportions preserved across train/val/test splits via stratification	1. Use 5-class dataset with equal 200-sample classes 2. Run split 3. Compare per-class counts across splits	Each class appears in all three splits in proportion ±2 samples; no class missing from any split	Critical	Positive
TC031	Data Split	Label Mapping	label2id and id2label dictionaries built correctly from sorted unique category values	1. Provide labeled CSV with categories: software, hardware, network 2. Run train_stage2_classifier() 3. Inspect label2id output	label2id = {'hardware':0,'network':1,'software':2}; id2label is its exact inverse	Critical	Positive
TC032	Data Split	Class Weights	Class weights computed as 'balanced' inverse-frequency weights	1. Use imbalanced dataset (class A: 800 samples, class B: 200 samples) 2. Run train_stage2_classifier() 3. Inspect printed class_weights dict	Weight for class B is ~4x weight for class A; weights sum to approximately num_classes	High	Positive
TC033	Data Split	Weighted Loss	CrossEntropyLoss instantiated with class_weights_tensor and label_smoothing	1. Run train_stage2_classifier() with label_smoothing=0.10 2. Inspect loss_fn object in debug mode	loss_fn.weight equals class_weights_tensor; loss_fn.label_smoothing == 0.10	High	Positive
TC034	Data Split	Split Summary Print	Console prints formatted data split table with per-class distribution across all splits	1. Run train_stage2_classifier() 2. Observe stdout	Table printed with header 'DATA SPLIT SUMMARY'; per-class rows shown for Train/Val/Test columns	Medium	Positive
TC035	Data Split	Seed Reproducibility	Same random seed produces identical splits on two separate runs	1. Run train_stage2_classifier() twice with seed=42 2. Compare train_df index sets	Identical row indices in train/val/test splits both runs; no randomness between runs	High	Positive

TC036	Data Split	Single Class Input	Pipeline raises informative error when labeled CSV has only one unique category	1. Prepare CSV with all rows having category='hardware' 2. Run train_stage2_classifier() 3. Observe exception	Stratified split fails with clear error; message indicates insufficient class diversity	Medium	Negative
TC037	Data Split	label2id Persistence	label2id.json and id2label.json saved to output_dir when best model checkpoint saved	1. Run train_stage2_classifier() 2. Inspect output_dir after training	label2id.json and id2label.json present; contents match label2id/id2label dicts used in training	Critical	Positive
TC038	Data Split	DataLoader Shape	LabeledTicketDataset DataLoader yields batches with correct keys and shapes	1. Create LabeledTicketDataset with 32 items, max_length=128 2. Create DataLoader with batch_size=16 3. Inspect first batch	Batch contains keys input_ids, attention_mask, labels; input_ids shape [16, 128]; labels shape [16]	High	Positive

Stage 2 – Classifier Fine-Tuning (TC039 – TC056)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC039	Stage 2 FT	Model Initialisation	RobertaForSequenceClassification loaded from Stage 1 checkpoint with correct num_labels	1. Save a toy Stage 1 model 2. Call from_pretrained() with num_labels=5 3. Inspect model.config	model.config.num_labels == 5; label2id and id2label stored in config; no weight mismatch error	Critical	Positive
TC040	Stage 2 FT	Forward Pass	Single forward pass on training batch produces logits of correct shape	1. Create batch with batch_size=16, max_length=128 2. Run model forward pass 3. Inspect outputs.logits	logits shape [16, num_classes]; no NaN values; no shape errors	Critical	Positive
TC041	Stage 2 FT	Loss Backward	Backward pass computes gradients for all model parameters without NaN	1. Run forward pass + loss.backward() on one batch 2. Check parameter gradients	All parameters have .grad attribute set; no NaN or Inf gradients after clip	Critical	Positive
TC042	Stage 2 FT	Training Metrics Print	Each epoch prints train and val Loss, Acc, W-F1, M-F1 in formatted block	1. Run train_stage2_classifier() for 2 epochs 2. Capture stdout	Output contains 'Train →' and 'Val →' lines with Loss, Acc, W-F1, M-F1 per epoch; fit_status line shown	High	Positive
TC043	Stage 2 FT	Checkpoint Saving	Best model checkpoint saved whenever val_loss improves	1. Run train_stage2_classifier() 2. Observe '✅ Saved best model' messages 3. Confirm files in output_dir	Best checkpoint saved at least once; config.json present; saved val_loss matches printed value	Critical	Positive
TC044	Stage 2 FT	Early Stopping	Training halts after 'patience' epochs without val_loss improvement	1. Set patience=2 2. Monkeypatch model to return constant val_loss after epoch 1 3. Run training	'🛑 Early stopping triggered.' printed after epoch 3; training loop exits; no additional epochs	Critical	Positive
TC045	Stage 2 FT	Fallback Checkpoint	Fallback save triggered if no checkpoint was saved during entire training run	1. Monkeypatch evaluate_classifier to always return higher loss than initial 2. Run training 3. Check output_dir	'⚠ No checkpoint saved. Saving current model as fallback.' printed; config.json present in output_dir	High	Positive
TC046	Stage 2 FT	Overfit Warning	'⚠ overfit' status printed when val_loss - train_loss gap exceeds 0.15	1. Monkeypatch evaluate_classifier to return val_loss 0.30 higher than train_loss 2. Observe epoch output	Fit status line contains '⚠ overfit gap=' followed by the gap value rounded to 4 decimals	Medium	Positive
TC047	Stage 2 FT	Mild Overfit Warning	'mild overfit' status printed when gap is between 0.05 and 0.15	1. Monkeypatch evaluate_classifier to return val_loss 0.10 above train_loss 2. Observe output	'mild overfit gap=' printed; no '⚠' symbol; gap value shown	Medium	Positive

TC048	Stage 2 FT	Good Fit Status	'good fit' status printed when val/train gap is <= 0.05	1. Monkeypatch evaluate_classifier to return val_loss 0.02 above train_loss 2. Observe output	'good fit gap=' printed; no overfit warning; gap value <= 0.05	Medium	Positive
TC049	Stage 2 FT	Improvement Counter	epochs_no_improve counter resets to 0 when val_loss improves	1. Run training where epoch 1 improves, epoch 2 worsens, epoch 3 improves again 2. Check counter via logs	Counter resets to 0 on epoch 3 improvement; 'No improvement (1/patience)' shown only for epoch 2	High	Positive
TC050	Stage 2 FT	History Accumulation	train_history list accumulates one dict per completed epoch with all metric keys	1. Run train_stage2_classifier() for 3 epochs 2. Inspect returned train_history	len(train_history) == 3; each dict has keys: epoch, train_loss, train_acc, train_wf1, train_mf1, val_loss, val_acc, val_wf1, val_mf1	High	Positive
TC051	Stage 2 FT	Return Tuple	train_stage2_classifier() returns (output_dir, label2id, id2label, train_history) tuple	1. Capture return value 2. Unpack tuple 3. Check types	stage2_dir is str; label2id is dict; id2label is dict; train_history is list; no ValueError on unpack	High	Positive
TC052	Stage 2 FT	Accuracy Monotonicity	Training accuracy generally increases over 10 epochs on a learnable dataset	1. Run 10 epochs on a 5-class balanced dataset 2. Compare train_acc in epoch 1 vs epoch 10 from history	Epoch 10 train_acc > Epoch 1 train_acc; no epoch shows train_acc == 0	High	Positive
TC053	Stage 2 FT	Gradient Clip Stage 2	Gradient clipping applied in Stage 2 training loop to max_norm=1.0	1. Hook gradient norms during one training epoch 2. Verify clip is applied	clip_grad_norm_ called each step with max_norm=1.0; no exploding gradient (loss stays finite)	High	Positive
TC054	Stage 2 FT	Scheduler Steps	Cosine LR scheduler steps once per training batch, not per epoch	1. Log optimizer.param_groups[0]['lr'] each batch for one epoch 2. Verify lr changes per-step	LR changes at each batch step; not held constant for full epoch; cosine decay visible	Medium	Positive
TC055	Stage 2 FT	Label Smoothing Effect	label_smoothing=0.0 produces higher confidence scores than label_smoothing=0.1	1. Train two models: one with label_smoothing=0.0, one with label_smoothing=0.1 2. Compare average max-softmax probabilities on validation set	Model with smoothing=0.0 has higher average confidence; smoothing model shows more calibrated output	Medium	Positive
TC056	Stage 2 FT	Tokenizer Inheritance	Stage 2 tokenizer loaded from Stage 1 checkpoint, not fresh roberta-base	1. Inspect tokenizer loaded in train_stage2_classifier() 2. Compare vocab to fresh roberta-base	Tokenizer loaded from stage1_model_dir path; vocab identical to Stage 1 saved tokenizer	High	Positive

Evaluation & Metrics (TC057 – TC073)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC057	Evaluation	evaluate_classifier()	evaluate_classifier() returns correct loss, accuracy, f1, weighted_f1, macro_f1	1. Create mock model returning known logits 2. Create loader with known labels 3. Call evaluate_classifier() 4. Verify each metric	loss, accuracy, f1, weighted_f1, macro_f1 all present; values match manual calculations for given logits/labels	Critical	Positive
TC058	Evaluation	Preds & Labels Return	evaluate_classifier() returns full preds and labels lists	1. Call evaluate_classifier() on a known 50-sample loader 2. Check 'preds' and 'labels' keys	preds and labels each have length 50; all values are valid class indices	High	Positive
TC059	Evaluation	No Gradient Leak	evaluate_classifier() runs under torch.no_grad() to prevent gradient accumulation	1. Inspect source or patch torch.no_grad 2. Confirm it wraps the evaluation loop	No gradients computed during evaluation; model.training == False during evaluation	High	Positive

TC060	Evaluation	Final Eval – All Splits	Final best-model evaluation runs on train, val, and test splits	1. Run full train_stage2_classifier() 2. Check printed final evaluation table	Table shows Train, Validation, and Test rows with Loss, Acc, W-F1, M-F1 for each split	Critical	Positive
TC061	Evaluation	Classification Report	Classification report saved as JSON for each of train, val, test splits	1. Run train_stage2_classifier() 2. Check output directory for JSON files	classification_report_training.json, classification_report_validation.json, classification_report_testing.json all present	Critical	Positive
TC062	Evaluation	Classification Report	JSON classification report contains precision, recall, f1-score per class	1. Load classification_report_testing.json 2. Inspect keys for each class	Each class key contains 'precision', 'recall', 'f1-score', 'support'; values are floats in [0,1]	High	Positive
TC063	Evaluation	Confusion Matrix PNG	Confusion matrix PNG saved for each of training, validation, testing splits	1. Run train_stage2_classifier() 2. Check plots directory	confusion_matrix_training.png, confusion_matrix_validation.png, confusion_matrix_testing.png all present; files > 0 bytes	Critical	Positive
TC064	Evaluation	Confusion Matrix JSON	Confusion matrix saved as JSON with 'labels' and 'matrix' keys	1. Load confusion_matrix_testing.json 2. Inspect structure	'labels' key is list of class names; 'matrix' key is NxN list of ints; rows sum to per-class test counts	High	Positive
TC065	Evaluation	Learning Curves PNG	Learning curves PNG saved to plots directory after training	1. Run train_stage2_classifier() 2. Check plots directory	stage2_learning_curves.png present; file size > 10 KB indicating non-empty plot	High	Positive
TC066	Evaluation	eval_results.json	eval_results.json saved with all required top-level keys	1. Run train_stage2_classifier() 2. Load eval_results.json 3. Check keys	Keys present: model, num_classes, class_names, data_split, best_val_loss, train_metrics, val_metrics, test_metrics, per_class_training_metrics, per_class_validation_metrics, per_class_testing_metrics, train_history, hyperparameters	Critical	Positive
TC067	Evaluation	eval_results.json	Hyperparameters block in eval_results.json matches values passed to trainer	1. Run train_stage2_classifier() with lr=1e-5, batch_size=8 2. Load eval_results.json 3. Check hyperparameters block	learning_rate == 1e-5 and batch_size == 8 in JSON; all other hyperparameter keys present	High	Positive
TC068	Evaluation	Test Accuracy Threshold	Test accuracy >= 0.80 on a balanced synthetic dataset with clear category signals	1. Generate 1000-sample balanced 5-class dataset with distinct vocabulary per class 2. Run full pipeline 3. Read test accuracy from eval_results.json	test_metrics.accuracy >= 0.80; no class with F1 < 0.60	Critical	Positive
TC069	Evaluation	Macro F1 vs Weighted F1	Macro F1 and weighted F1 differ on imbalanced dataset (both reported correctly)	1. Use imbalanced 5-class dataset 2. Run evaluation 3. Compare macro_f1 and weighted_f1 values	macro_f1 != weighted_f1 on imbalanced data; weighted_f1 closer to accuracy; both in eval_results	Medium	Positive
TC070	Evaluation	Report Per-Class Print	Classification report printed to console for each split with target_names labels	1. Run train_stage2_classifier() 2. Capture stdout 3. Search for 'TRAINING Classification Report', 'VALIDATION', 'TESTING'	Three separate report blocks in stdout each preceded by split name header and sample count	High	Positive
TC071	Evaluation	Metric Rounding	All metrics in eval_results.json rounded to 6 decimal places	1. Load eval_results.json 2. Check decimal precision of accuracy, f1, weighted_f1, macro_f1, loss	Values have at most 6 decimal digits; no scientific notation; values in valid ranges	Low	Positive

TC072	Evaluation	Per-Class Keys	per_class_testing_metrics contains an entry for every class in ordered_names	1. Load eval_results.json 2. Compare per_class_testing_metrics keys against class_names	per_class_testing_metrics has exactly num_classes entries; each entry has precision, recall, f1, support	High	Positive
TC073	Evaluation	Confusion Matrix Sanity	Row sums of confusion matrix equal per-class sample counts in test split	1. Load confusion_matrix_testing.json 2. Compute row sums 3. Compare to test split per-class counts	Each row i sums to number of test samples with true label i; total sum equals test set size	High	Positive

Inference & Prediction (TC074 – TC086)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC074	Inference	load_predictor()	load_predictor() returns tokenizer, model, and id2label from saved checkpoint	1. Save a trained checkpoint 2. Call load_predictor(model_dir) 3. Inspect return types	tokenizer is RobertaTokenizerFast; model is RobertaForSequenceClassification; id2label is dict with int keys	Critical	Positive
TC075	Inference	Model in Eval Mode	Model returned by load_predictor() is in eval mode (model.training == False)	1. Call load_predictor() 2. Check model.training attribute	model.training == False; calling predict does not toggle training mode back to True	High	Positive
TC076	Inference	predict_ticket() Output	predict_ticket() returns dict with required keys for a valid ticket description	1. Load predictor 2. Call predict_ticket('VPN login keeps failing') 3. Inspect returned dict	Dict contains: text, predicted_category, confidence_score, entropy, margin; all fields non-empty	Critical	Positive
TC077	Inference	Category in Known Set	predicted_category is always one of the trained class names	1. Call predict_ticket() on 50 diverse ticket texts 2. Check predicted_category for each	All predicted_category values are in id2label.values(); no unknown or null categories returned	Critical	Positive
TC078	Inference	Confidence Score Range	confidence_score is always a float in [0.0, 1.0]	1. Call predict_ticket() on 50 diverse ticket texts including noise 2. Check confidence_score values	All confidence_score values are floats; 0.0 <= confidence_score <= 1.0 for every prediction	Critical	Positive
TC079	Inference	Entropy Calculation	Entropy is non-negative and higher for uncertain predictions	1. Predict on clear-category ticket vs noisy/garbled text 2. Compare entropy values	entropy >= 0 for all inputs; garbled text produces higher entropy than clear ticket text	High	Positive
TC080	Inference	Margin Calculation	Margin = top1_prob - top2_prob; always non-negative	1. Call predict_ticket() on a highly ambiguous ticket 2. Call on clear-category ticket 3. Compare margin values	margin >= 0 for all inputs; clear ticket has larger margin; ambiguous ticket has margin near 0	High	Positive
TC081	Inference	Rounding	confidence_score, entropy, margin all rounded to 4 decimal places	1. Call predict_ticket() 2. Convert confidence_score to string and count decimal digits	confidence_score, entropy, margin each have at most 4 decimal digits in returned dict	Medium	Positive
TC082	Inference	Truncation	Predict on ticket longer than max_length=128 tokens without error	1. Construct ticket description of 500 words 2. Call predict_ticket() 3. Check output	predict_ticket() returns valid result; no IndexError or shape mismatch; truncation applied silently	High	Positive
TC083	Inference	Empty Input	predict_ticket() handles empty string input without exception	1. Call predict_ticket('') 2. Observe output or exception	Returns dict with valid predicted_category; low confidence expected; no crash or KeyError	High	Negative

TC084	Inference	Noisy Input	predict_ticket() returns valid category for garbled/all-OOV text	1. Input: 'asdfjkl qwerty 12345 !!!' 2. Call predict_ticket() 3. Inspect output	predicted_category is a known class name; no exception; low confidence and high entropy expected	High	Negative
TC085	Inference	Inference Speed	Single ticket inference completes in under 200ms on CPU	1. Load predictor on CPU 2. Time 100 individual predict_ticket() calls 3. Compute average	Average inference time < 200ms per ticket on CPU; suitable for real-time help-desk routing	High	Performance
TC086	Inference	Batch Prediction	predict_ticket() called in loop over 100 tickets produces 100 valid JSON dicts	1. Prepare list of 100 ticket descriptions 2. Loop and call predict_ticket() for each 3. Check output list length and structure	100 dicts returned; each has all required keys; no exceptions during batch; total time < 20s	High	Positive

Pipeline Integration (TC087 – TC095)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC087	Pipeline	End-to-End Flow	Full pipeline runs from raw CSVs through Stage 1, Stage 2, and prediction without errors	1. Provide source CSVs and labeled CSV 2. Run complete pipeline: prepare_unlabeled_csv → train_stage1_mlm → train_stage2_classifier → load_predictor → predict_ticket 3. Inspect final output	Pipeline completes without exceptions; eval_results.json saved; sample predictions printed	Critical	Positive
TC088	Pipeline	Stage 1 → Stage 2 Link	Stage 2 loads tokenizer/model from Stage 1 output directory	1. Run Stage 1 with output_dir='./stage1_out' 2. Pass './stage1_out' as stage1_model_dir to Stage 2 3. Verify no re-download of roberta-base	Stage 2 loads from './stage1_out'; no fresh download from HuggingFace hub during Stage 2 init	Critical	Positive
TC089	Pipeline	Artifact Completeness	All expected artifacts present in OUTPUT_DIR after full pipeline run	1. Run full pipeline 2. List roberta_2stage_output/ and plots/ subdirectory	eval_results.json, 3x classification_report_*.json, 3x confusion_matrix_*.json, stage2_learning_curves.png, 3x confusion_matrix_*.png all present	Critical	Positive
TC090	Pipeline	Inference-Only Mode	Prediction runs from saved artifacts without retraining	1. Run full pipeline once 2. Call load_predictor() and predict_ticket() on new text without re-running training 3. Verify output	Valid prediction returned; no training steps executed; inference < 1 second per ticket	High	Positive
TC091	Pipeline	Preprocessing Consistency	Tokenizer applied identically at training time and prediction time	1. Tokenize a known sentence during Stage 2 training 2. Tokenize the same sentence via load_predictor() tokenizer 3. Compare token IDs	Identical token ID sequences both times; no discrepancy due to fresh re-initialisation	Critical	Positive
TC092	Pipeline	Config Parameterisation	Hyperparameters (lr, batch_size, epochs) passed as arguments, not hardcoded in functions	1. Call train_stage1_mlm() with lr=1e-4, batch_size=32 2. Verify training uses those values 3. Repeat for train_stage2_classifier()	AdamW lr matches argument; DataLoader batch_size matches argument; no override by internal defaults	High	Positive
TC093	Pipeline	set_seed() Effect	set_seed(42) produces reproducible training results across two identical runs	1. Run full Stage 2 training twice with same data and set_seed(42) 2. Compare train_history from both runs	train_loss and val_loss values identical in both runs for every epoch	High	Positive
TC094	Pipeline	Error Propagation	Corrupt input CSV in Stage 2 halts pipeline with informative error	1. Pass a non-CSV binary file as labeled_csv to train_stage2_classifier() 2. Observe error handling	Exception raised with message identifying file parse failure; no partial model saved; no silent continuation	High	Negative

TC095	Pipeline	Output Dir Creation	OUTPUT_DIR and PLOTS_DIR created automatically at startup if they do not exist	1. Delete roberta_2stage_output/ if present 2. Run pipeline 3. Verify directory creation	roberta_2stage_output/ and roberta_2stage_output/plots/ created; os.makedirs with exist_ok=True prevents errors	Medium	Positive

Security & Data Leakage (TC096 – TC098)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC096	Security	No Test-Label Leakage	Test set labels not used during training; test_loader not passed to optimizer loop	1. Instrument train_stage2_classifier() to check which loaders use backward() 2. Verify test_loader is excluded	backward() never called with test_loader batches; test split evaluated only after best model loaded	Critical	Security
TC097	Security	No Val Tokenizer Fit	Tokenizer fitted only on training texts; validation and test encoded with frozen vocab	1. Inspect tokenizer usage: tokenizer.fit() or tokenizer.train() should not be called on val/test texts 2. Verify stage2 uses pre-trained tokenizer throughout	RobertaTokenizerFast used as-is (pre-trained vocab); no vocabulary expansion from val/test data	Critical	Security
TC098	Security	No Class-Weight Leak	Class weights computed from training labels only, not from val or test labels	1. Inspect compute_class_weight() call in train_stage2_classifier() 2. Verify only train_labels_np is passed	compute_class_weight called with y=train_df['label_id'].values exclusively; val/test labels not used	High	Security

Performance (TC099 – TC104)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC099	Performance	Stage 1 Training Speed	Stage 1 MLM trains 3 epochs on 5000-sample unlabeled CSV in under 15 minutes on CPU	1. Generate 5000-row unlabeled CSV 2. Run train_stage1_mlm() with batch_size=16, epochs=3 3. Measure wall-clock time	Total Stage 1 time < 15 minutes on CPU; no memory allocation errors; perplexity computed	Medium	Performance
TC100	Performance	Stage 2 Training Speed	Stage 2 fine-tuning runs 10 epochs on 1000 labeled samples in under 10 minutes on CPU	1. Use 1000-row balanced labeled CSV 2. Run train_stage2_classifier() with batch_size=16, epochs=10 3. Measure wall-clock time	Total Stage 2 time < 10 minutes on CPU; early stopping may reduce actual epochs	Medium	Performance
TC101	Performance	Inference Throughput	100 sequential predict_ticket() calls complete in under 20 seconds on CPU	1. Load predictor 2. Time 100 sequential predict_ticket() calls using time.perf_counter() 3. Compute total and average	Total time < 20 seconds; average per-ticket < 200ms; no degradation over repeated calls	High	Performance
TC102	Performance	Memory – Stage 1	Peak memory during Stage 1 training stays below 4 GB on CPU with batch_size=16	1. Run Stage 1 with batch_size=16 on 5000 texts 2. Monitor peak RAM with tracemalloc or psutil	Peak memory < 4 GB; memory released between epochs; no OOM error	Medium	Performance
TC103	Performance	Memory – Stage 2	Peak memory during Stage 2 training stays below 4 GB on CPU with batch_size=16	1. Run Stage 2 on 1000 labeled samples, batch_size=16 2. Monitor peak RAM	Peak memory < 4 GB; memory released after each epoch; no memory leak across epochs	Medium	Performance
TC104	Performance	GPU Acceleration	Training 3x faster on CUDA GPU compared to CPU baseline for Stage 2	1. Time Stage 2 epoch on CPU 2. Time same epoch on CUDA GPU 3. Compute ratio	GPU epoch time < CPU epoch time / 3; device=cuda used for model and tensors; no dtype errors	Low	Performance

Robustness (TC105 – TC110)

TC ID	Module	Feature	Description	Test Steps	Expected Result	Priority	Type
TC105	Robustness	Noise Tolerance	predict_ticket() returns valid output for garbled/all-symbol input	1. Input: '!@#$%^&*() 123 ??? ' 2. Call predict_ticket() 3. Inspect output	Valid dict returned; predicted_category is known class; no exception; entropy expected to be high	High	Positive
TC106	Robustness	Single-Class Batch	Loss computation does not crash when a training batch contains only one class	1. Create loader where all samples are class 0 2. Run one training step 3. Check loss value	Loss computed successfully; no division-by-zero; no NaN; gradients flow normally	High	Positive
TC107	Robustness	Very Long Description	Tokenizer truncates 1000-word description to max_length without shape mismatch	1. Construct 1000-word ticket description 2. Tokenize with max_length=128 3. Run forward pass	input_ids shape [1, 128]; no error; forward pass returns valid logits	High	Positive
TC108	Robustness	Minimal Dataset	Pipeline runs without error on a dataset with exactly 10 labeled samples (2 per class, 5 classes)	1. Create 10-row labeled CSV 2. Run train_stage2_classifier() 3. Check for errors	Pipeline completes or raises a clear minimum-data error; no silent crash or NaN loss	Medium	Negative
TC109	Robustness	Unicode / Emoji Input	predict_ticket() handles Unicode emojis in ticket description without crash	1. Input: 'My laptop 💻 is not connecting to WiFi 📶' 2. Call predict_ticket() 3. Inspect output	Valid prediction returned; no UnicodeDecodeError; emojis tokenized or skipped by BPE tokenizer	Medium	Positive
TC110	Robustness	Repeated Prediction Stability	Calling predict_ticket() 100 times on the same text produces identical results	1. Call predict_ticket() 100 times on 'Printer not responding' 2. Compare all outputs	Identical predicted_category, confidence_score, entropy, margin for all 100 calls; deterministic output	Medium	Positive
TC111	Data Ingestion	CSV Format Enforcement	prepare_unlabeled_csv and load_labeled_data strictly enforce CSV format; non-CSV files raise ValueError	1. Try to pass a .txt file to prepare_unlabeled_csv() 2. Try to pass the same .txt to load_labeled_data() 3. Observe exceptions	Both functions raise ValueError with message "Only CSV format is supported"	High	Negative
TC112	Documentation	Docstrings	Docstrings present for save_json, load_unlabeled_texts, load_labeled_data, predict_ticket	1. Inspect docstrings of these four functions using inspect.getdoc() 2. Verify each is non-empty	All four functions contain valid descriptive docstrings	Medium	Positive
TC113	Inference	Input/Output Assertions	Assertions in predict_ticket and train_stage2_classifier for input shape and output shape	1. Source code check: assert statements present 2. Runtime check: force mismatched logits shape in predict_ticket	AssertionError raised with message about num_classes mismatch	Critical	Positive
TC114	Pipeline Integration	No config.yaml Dependency	Pipeline runs via function Default args, so no config.yaml needed	1. Run train_stage2_classifier() with explicit lr, batch_size, epochs args (no config.yaml present)	Pipeline runs successfully without FileNotFoundError	High	Positive

— End of Document —